

jnj-deltashare

A little [Delta Sharing](#) client that reads data shared with us from the `franks_cdl` share on the [e2-demo-field-eng](#) Databricks workspace.

Delta Sharing/Open Sharing is an open protocol for secure data sharing. We connect as an **open (token-based) recipient**: Databricks gives the recipient an *activation link*, which downloads a credential file (`config.share`) containing the sharing server endpoint and a bearer token. The client reads that file — no Databricks account required on our side.

Source share

Workspace	<code>e2-demo-field-eng</code>
Share	<code>franks_cdl</code>
Tables	<code>ai_agent.customers</code> , <code>ai_agent.billing</code> (and any others added to the share)

The `endpoint` inside your `config.share` points at the **regional Delta Sharing serving host** (e.g. `oregon.cloud.databricks.com`) that fronts this workspace's metastore — that's expected and not a different workspace. `e2-demo-field-eng` above is the browsable workspace where the share is managed.

What's in here

File	Purpose
<code>delta_sharing_client.ipynb</code>	The notebook client: installs deps, connects, lists shared tables, loads the first available table into pandas.
<code>config.share</code>	Your credential file — not committed. You must add this yourself (see below).
<code>.gitignore</code>	Keeps <code>*.share</code> and the venv out of git.

Provider-side setup (creating the share)

This is the **data provider's** side — done once in the Databricks workspace that owns the data. If you're only *consuming* an existing share, skip to [Setup](#) below. Run the SQL in a Unity Catalog-enabled workspace as a user with the `CREATE SHARE / CREATE RECIPIENT` metastore privileges.

Note: the screenshots below were captured from an earlier iteration when the share was named `jnj-butterfly-share`. The current share is `franks_cdl` — the steps are identical.

1. Create the share:

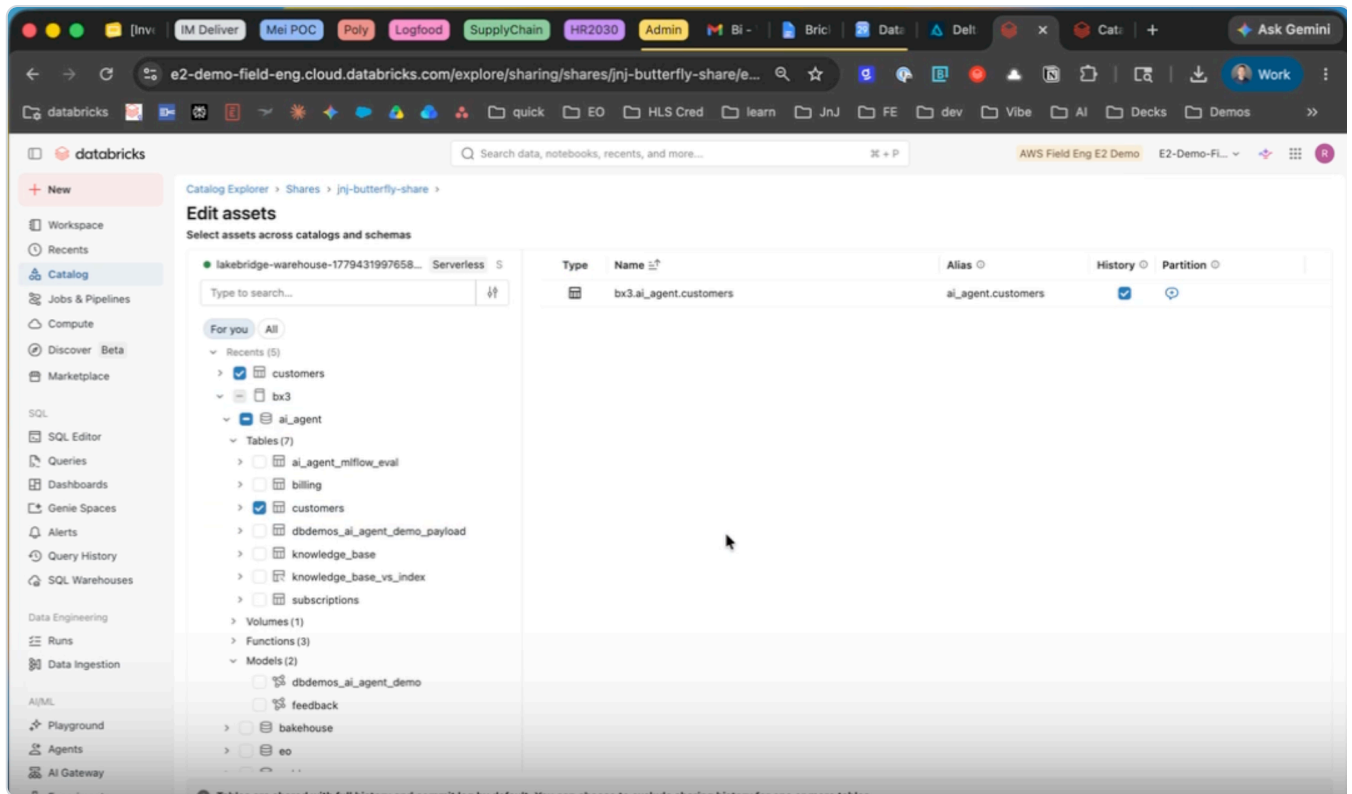
```
CREATE SHARE IF NOT EXISTS franks_cdl
COMMENT 'CDL demo data shared via Delta Sharing';
```

2. Add tables to the share (schema-qualified from a Unity Catalog catalog):

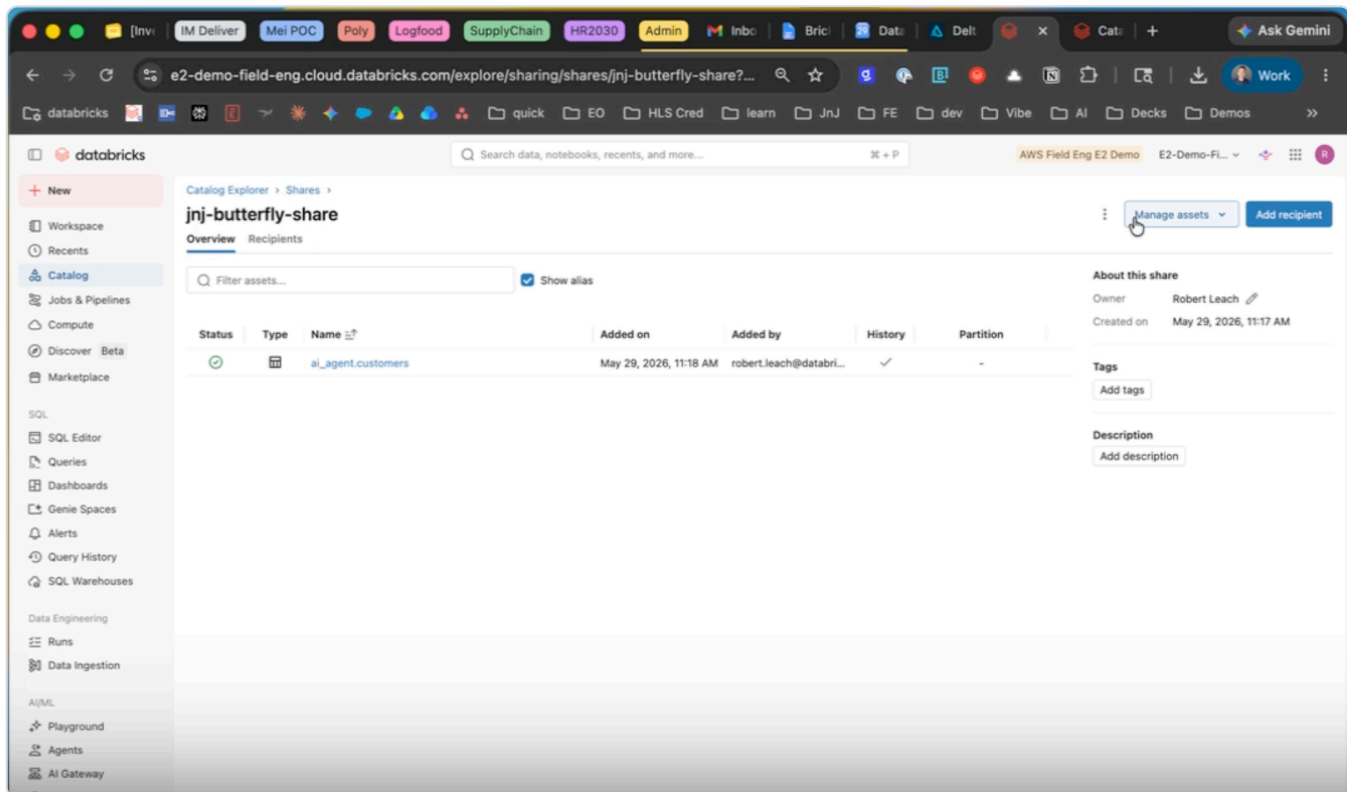
```
ALTER SHARE franks_cdl ADD TABLE ai_agent.customers;
ALTER SHARE franks_cdl ADD TABLE ai_agent.billing;
```

Optionally enable history so recipients can time-travel:

```
ALTER SHARE franks_cdl ADD TABLE ai_agent.customers WITH HISTORY;
```



In the UI, **Manage assets** → **Edit assets** lets you pick tables from a catalog/schema tree, set an alias, and toggle History per table.



The share's **Overview** tab then lists each added asset (here `ai_agent.customers`, with History enabled).

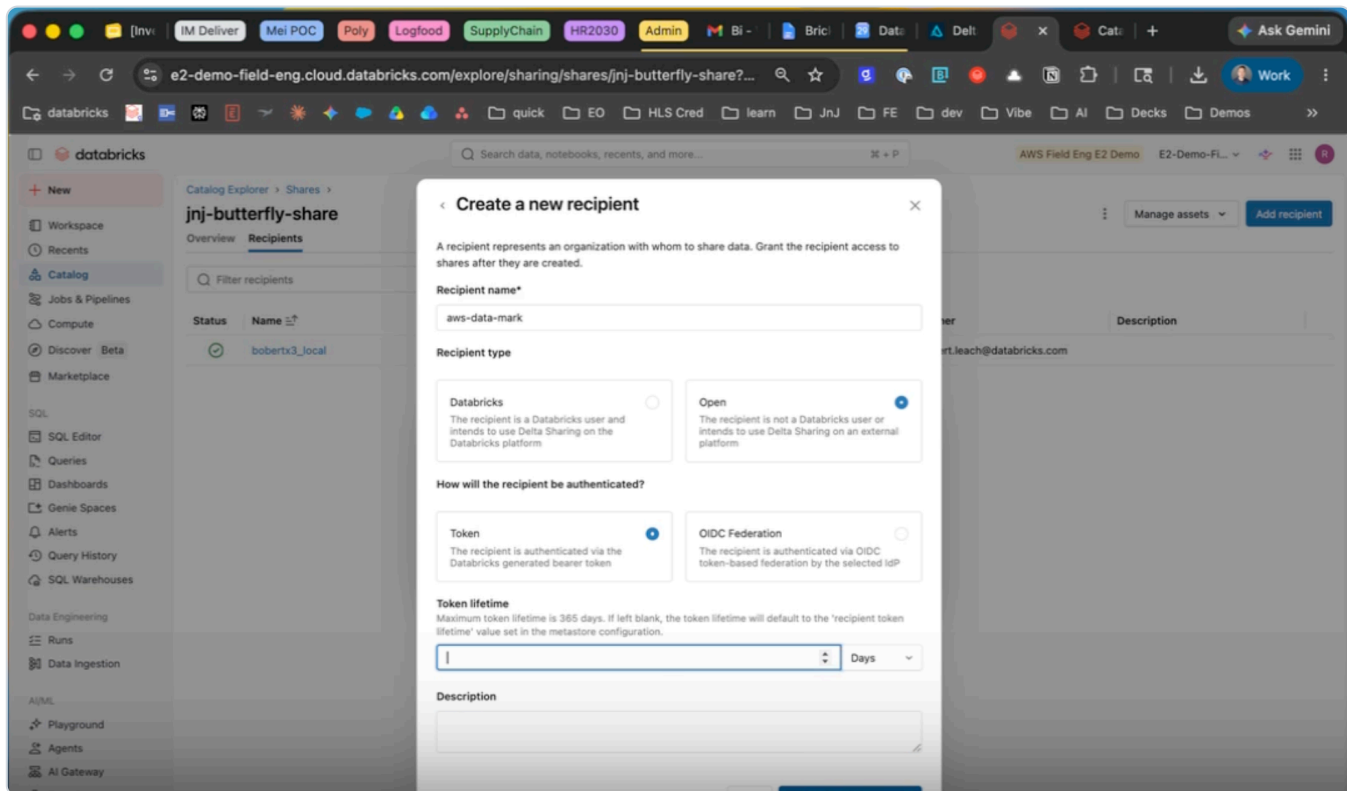
3. Create the open (token-based) recipient:

```
CREATE RECIPIENT IF NOT EXISTS bobertx3_local
COMMENT 'Open recipient for local Delta Sharing client';
```

This mints a single-use **activation link**. Retrieve it with:

```
DESCRIBE RECIPIENT bobertx3_local; -- see activation_link + token expiry
```

(You can also do all of this in Catalog Explorer → **Delta Sharing** → **Shared by me** and **Recipients**.)



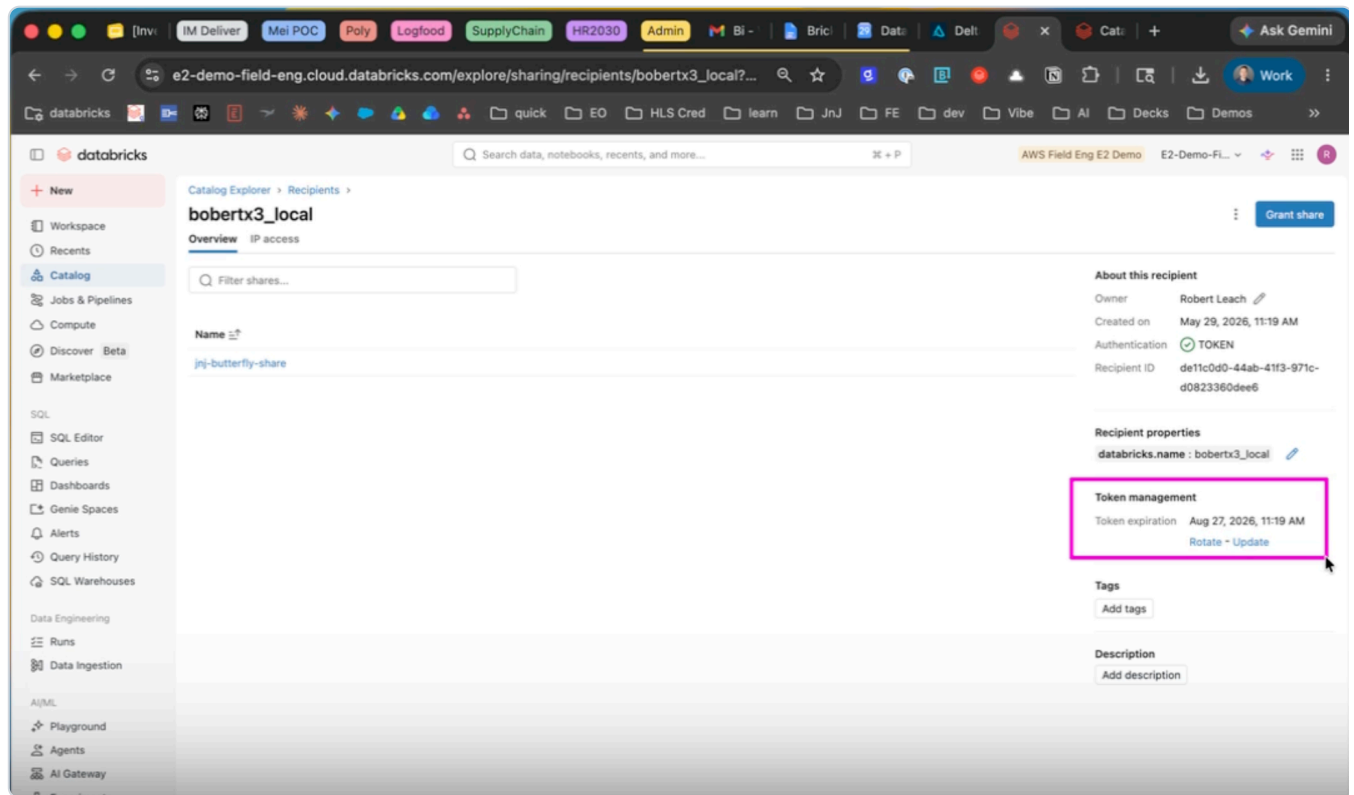
The **Add recipient** dialog: create an **Open** recipient authenticated by a Databricks-generated **Token**, with an optional token lifetime.

4. Grant the recipient access to the share:

```
GRANT SELECT ON SHARE franks_cd1 TO RECIPIENT bobertx3_local;
```

5. Send the activation link to the recipient. They download `config.share` from it (step 1 below). To rotate a lost/expired token:

```
ALTER RECIPIENT bobertx3_local ROTATE TOKEN;
```



The recipient's page shows its granted shares, authentication type, and **Token management** — where the activation link lives and where you **Rotate** or **Update** the token.

Setup

1. Get the credential file

In the [e2-demo-field-eng workspace](#), open Catalog Explorer → **Recipients** → `bobertx3_local` :

1. Under **Token management**, copy/open the **Activation link** in a browser.
2. Click **Download credential file** — you'll get `config.share` .
3. Save it in this folder as `config.share` .

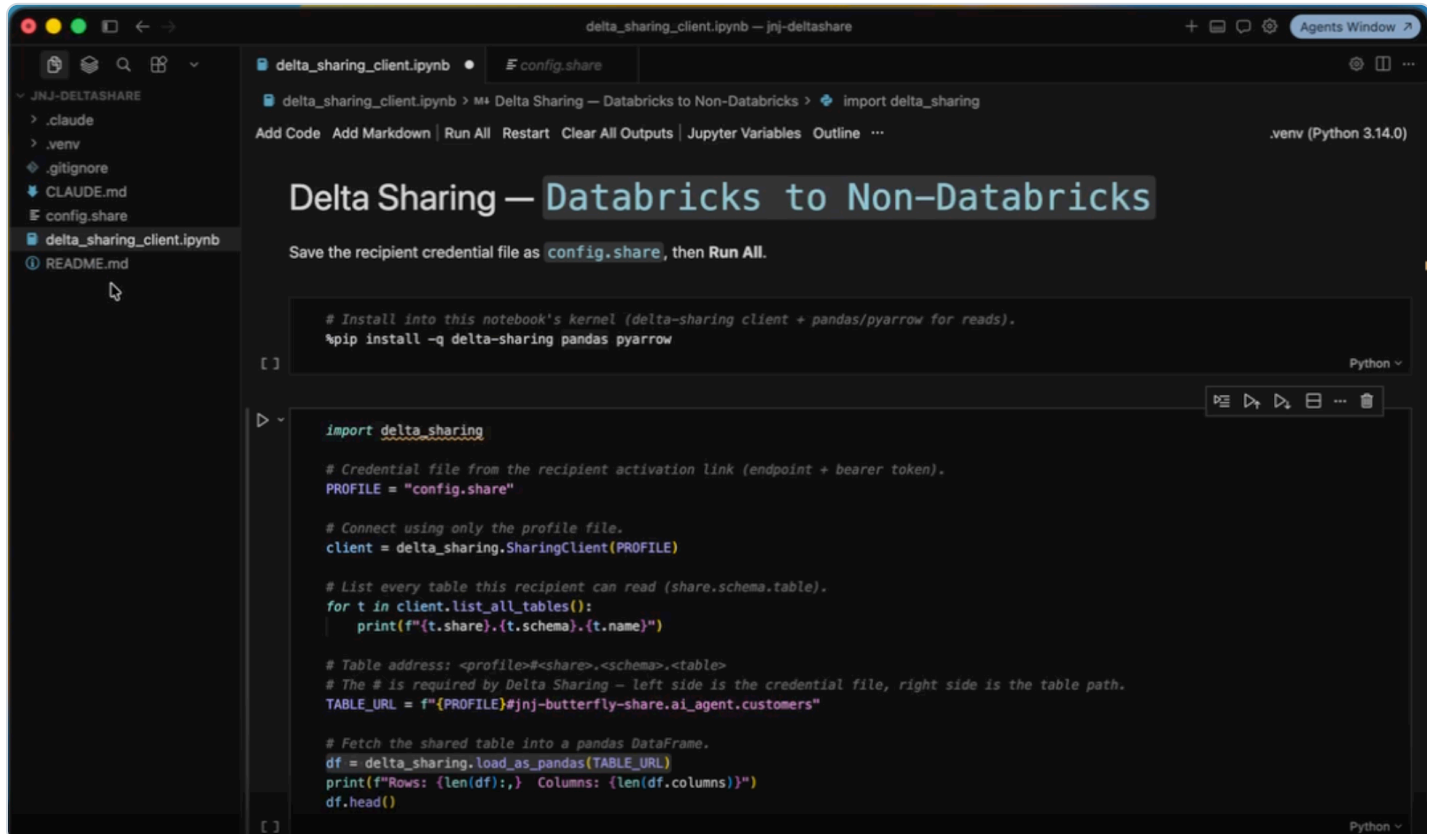
⚠ The activation link is **single-use**, and the token expires. Treat `config.share` like a password — it's git-ignored on purpose. If it's lost or compromised, rotate the recipient's token in Databricks and re-download.

2. Run the notebook

Open `delta_sharing_client.ipynb` in Jupyter or Cursor and **Run All**. The first code cell installs `delta-sharing`, `pandas`, and `pyarrow` into the active kernel.

Optional: use a local venv and Jupyter:

```
python3 -m venv .venv
./venv/bin/python -m pip install jupyter ipykernel delta-sharing pandas pyarrow
./venv/bin/jupyter notebook delta_sharing_client.ipynb
```



The client notebook (`delta_sharing_client.ipynb`): a `%pip install` cell, then connect with `SharingClient`, enumerate tables via `list_all_tables()`, and load one into pandas with `load_as_pandas()`.

How reading a table works

The credential file is referenced together with the table path using the format `<profile>#<share>.<schema>.<table>`. Share and table names come from `list_all_tables()` — nothing is hardcoded:

```
import delta_sharing

profile_file = "config.share"
```

```
client = delta_sharing.SharingClient(profile_file)

tables = client.list_all_tables()
for t in tables:
    print(f"{t.share}.{t.schema}.{t.name}")

t = tables[0]
table_url = f"{profile_file}#{t.share}.{t.schema}.{t.name}"
df = delta_sharing.load_as_pandas(table_url)
print(df.head())
```

Useful options:

- `delta_sharing.load_as_pandas(table_url, limit=100)` — peek at the first N rows.
- `delta_sharing.load_as_pandas(table_url, version=N)` — time travel (history must be enabled on the shared table).
- `delta_sharing.SharingClient(profile_file).list_all_tables()` — discover everything shared with this recipient.
- `delta_sharing.load_as_spark(table_url)` — read as a Spark DataFrame (requires PySpark + the `delta-sharing-spark` package).